Feedback survey

CS 340

I made this during COVID

Threading 0b10

forms.gle/7tjbrzQH4rH5gKcQ7

Updates

1. MP5 Allocator and MP6 Wallet Grading

2. MP6 Wallet out

w1 Spring break
w2 exam 2
w3 MP6 Due

3. Exam 2 sign up open
   a. Practice exam - by tomorrow end of day
   b. Practice coding - out now

4. No HW

Threading

Today's LGs - Expand your mental model of a program to include multiple concurrent threads.

Be able to identify simple race conditions and deadlocks

Be able to add mutexes, conditional variables, and reader/writer locks to a multithreaded program to avoid threading errors.

Agenda

1. Race Conditions
2. Mutex
3. Making data structures thread safe
4. Reader-writer locks
5. Condition Variables
6. Deadlock

Definitions

Race Condition - is when output varies based on which order threads run.

Race Conditions we care about -

causes a data race

makes your code not work as expected

Does this code have a race condition?

```
int balance = 0;

void *transaction(void *arg) {
    int delta = *((int*)arg);
    balance += delta;
    if(balance < 0) {
        //$10 fee for negative balance
        balance = -10;
    }
}

int main() {
 pthread_t threads[5];
 int transactions[5] = {40, -5, -10, -4, -10};

 for(int i = 0; i < 5; i++){
    pthread_create(&threads[i], NULL, transaction, &transactions[i]);
 }
 for(int i = 0; i < 5; i++){
    pthread_join(threads[i], NULL);
 }
 printf("Your balnce is now, %i", balance);
}
```

10

~~30~~

~~40~~

30

26

16

1

11

16

clicker.cs.illinois.edu

Q1

~Code~
340

Mutex

Definition - lock, only 1 thread at a time

Pattern

make mutex

get mutex
code
give back mutex

Static Mutex Example

```c
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t my_m = PTHREAD_MUTEX_INITIALIZER;

int shared_int = 0;

void *shared_function(void* arg){
    pthread_mutex_lock(&my_m);
    shared_int += 3;
    printf("my val: %i", shared_int);
    shared_int = 0;
    pthread_mutex_unlock(&my_m);
}

//thread creation code and main function
```

Static Vs. Dynamic Mutex

```c
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t my_m = PTHREAD_MUTEX_INITIALIZER;
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t* my_mutex;

void init_mutex(){
    my_mutex = malloc(sizeof(pthread_mutex_t));
    pthread_mutex_init(my_mutex, NULL);
}

void destroy_mutex(){
    pthread_mutex_destroy(my_mutex);
    free(my_mutex);
}

//code that inits the mutex, uses it, then destroys it
```

If I have an array of 5 items initialized at compile time and I want to protect each of them from race conditions could I use the static initialization method as shown below?

clicker.cs.illinois.edu

Q2

~Code~
340

yes static

```
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m3 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m4 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m5 = PTHREAD_MUTEX_INITIALIZER;
```

yes

no → dynamic

If I have a linked list data structure and I want to protect each datum from race conditions individually, could I use the static initialization method as shown below?

clicker.cs.illinois.edu

Q3

~Code~
340

```
pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m3 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m4 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t m5 = PTHREAD_MUTEX_INITIALIZER;
```

no

yes - static

no - dynamic

Linked List Example - Protecting the datum

```
typedef struct node {
    node * next;
    int datum
    pthread_mutex_t * m;
} node;
```

```
insert...(val)
    node * new_n = malloc(sizeof(node));
    new_n -> datum = val;
(1) new_n -> m = malloc(sizeof(pthread_mutex_t));
(2) pthread_mutex_init(new_n -> m, NULL);
```

pop

```
....
pthread_mutex_destory
    (old_n -> m);
free(old_n -> m);
....
free(old_n);
```

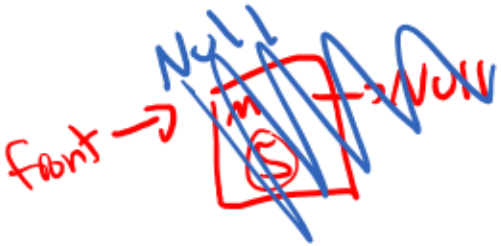
Could there be a race condition error?

```
typedef struct node {
    struct node *next;
    pthread_mutex_t *m;
    int datum;
} node;

typedef struct ll {
    node *front;
} ll;

void increment_front(ll *self){
    if(self->front != NULL) {
        pthread_mutex_lock(self->front->m);
        if(self->front->datum < 10){
            self->front->datum += 5;
        }
        pthread_mutex_unlock(self->front->m);
    }
}

//pop_front, init, destroy, push_front
```



reader lock

unlock reader

pop_front

clicker.cs.illinois.edu

Q4

~Code~
340

Making a Linked List Thread Safe

- we can't mess up the structure

option – lock down list every time we look or edit

Reader Writer Lock

Many can read, only 1 can write

lockA

3 threads get lock A to read

....

Thread want to edit, tries to get lockA in edit mode

Protecting the Structure Vs. the Data

1 reader writer lock

1 mutex per data

linked list with 5 nodes

1-reader_writer lock → protect pointers

5-mutexes → protect the datum

Reader Writer Lock Examples

```c
typedef struct node {
  struct node *next;
  //could add mutex
  int data;
} node;

typedef struct {
  node *head;
  pthread_rwlock_t *rwlock;
} my_list;

void my_list_init(my_list *list){
    list->head = NULL;
    list->rwlock = malloc(sizeof(pthread_rwlock_t));
    pthread_rwlock_init(list->rwlock, NULL);
}
```

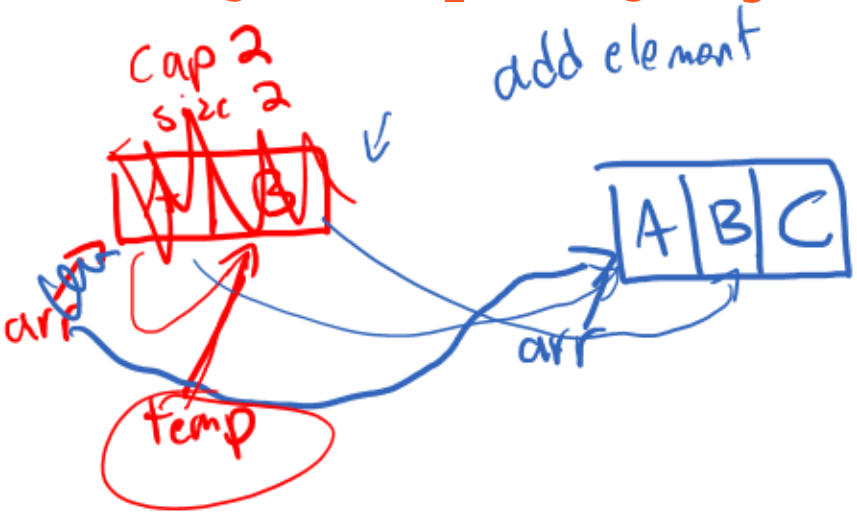
```c
int find_element(my_list *list, int index){
    pthread_rwlock_rdlock(list->rwlock);
    //read things
    pthread_rwlock_unlock(list->rwlock);
}

void add_element(my_list *list){
    pthread_rwlock_wrlock(list->rwlock);
    //change the structure of the list
    pthread_rwlock_unlock(list->rwlock);
}

void my_list_destroy(my_list *list){
    //delete all nodes and mutex's
    pthread_rwlock_destroy(list->rwlock);
}
```

free(list->rwlock)

Do you need synchronization tools to make a map thread safe? Why? Assume the map is implemented with a dynamic array of map entry objects.



clicker.cs.illinois.edu

Q5

~Code~
340

yes

Wallet

part-1 : make map thread safe

- reader writer lock

add/withdraw

reader lock

part 2 : write wallet_use

- account with money in the map
- amounts can't go negative

Condition Variable

Definition - lets a thread sleep until something is true

- lets threads communicate

Pattern

Make a mutex

Make a CV

Get the mutex lock

check predicate

- If false

wait on the CV

in the func

- release mutex
- sleep

- On signal

wake up

try to get Mutex

do something

signal other threads

signal to the CV

Static Condition Variable Example

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t my_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t my_cv = PTHREAD_COND_INITIALIZER;

int shared_int = 0;
int pred = 0;

void *shared_function(void* arg){
    pthread_mutex_lock(&my_mutex);
    while(pred == 0){
        pthread_cond_wait(&my_cv, &my_mutex);
    }
    shared_int = pred * 5;
    pred++;
    pthread_cond_broadcast(&my_cv);
    pthread_mutex_unlock(&my_mutex);
}

//thread creation code and main function, changes pred to 1 eventually
```

- release mutex
- sleep

get mutex

```c
void *first_computation(void *arg){
    pthread_mutex_lock(&my_mutex);
    printf("FIRST %i\n", shared_int);
    pred = 1;
    pthread_cond_broadcast(&my_cv);
    pthread_mutex_unlock(&my_mutex);
}
```

Static Vs. Dynamic Condition Variable

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t my_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t my_cv = PTHREAD_COND_INITIALIZER;

```

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

pthread_mutex_t my_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t* my_cv;

int shared_int = 0;
int pred = 0;

void init_cv(){
    my_cv = malloc(sizeof(pthread_cond_t));
    pthread_cond_init(my_cv, NULL);
}

void destroy_cv(){
    pthread_cond_destroy(my_cv);
    free(my_cv);
}

//code inits my_cv, runs code, then destroys cv
```

Deadlock

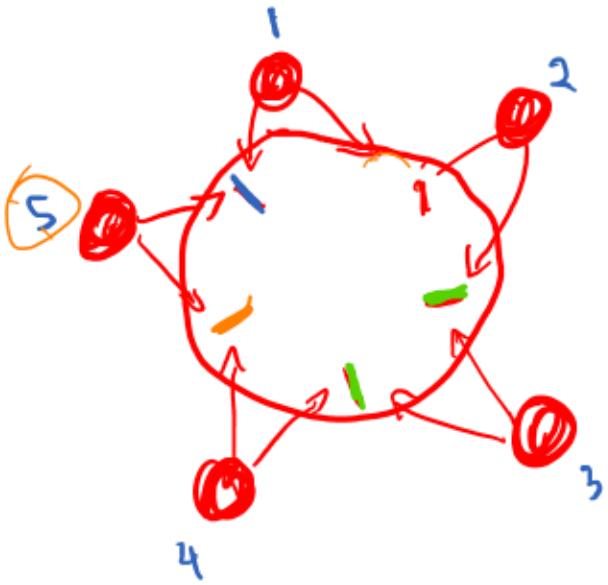
↑
expansion

Alice want to play
Bob the expansion

Alice – checks out
Ark Nova

Bob – check out
expansion

Deadlock - Dining Philosophers



eat - need Two chopsticks

think

pick up one at a time

What does deadlock look like in code?

MP6

1. reason through code
2. go to office hours
3. post on campus wire
4. rewrite code

real life

1. reason about code
2. rewrite code
3. remove threading

clicker.cs.illinois.edu

Q6

~Code~
340

your program pauses

Summary

Errors - race conditions
deadlock

Synchronization tools -

Mutex
reader/writer
condition variable

Let me know how things are going!